



Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

Laurea Magistrale in Sicurezza Informatica

Enea Manzi (65935A)

Relatore: Prof. Marco Anisetti

Correlatore: Prof. Claudio Agostino Ardagna

XX Luglio 2026

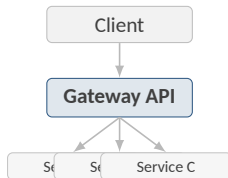


UNIVERSITÀ
DEGLI STUDI
DI MILANO



Contesto

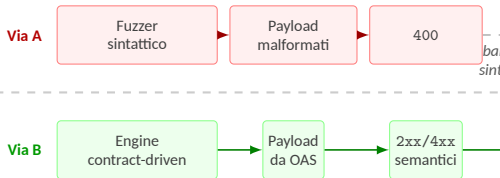
- Architetture a **microservizi**: la superficie di attacco cresce con le **interazioni tra servizi**, non con il loro numero
- Il **gateway API** come punto di enforcement centralizzato: autenticazione, autorizzazione, rate limiting
- Le vulnerabilità più rilevanti sono **semantiche**: richiedono la conoscenza del **comportamento atteso**
 - 4 delle prime 5 categorie OWASP API Security Top 10 2023





Stato dell'arte e gap

- **G1 Cecità semantica:** scanner e fuzzer operano sulla **sintassi** HTTP, non sulla **semantica** dei contratti
- **G2 Portabilità/profondità:** gli strumenti specializzati richiedono **riscrittura** per ogni target
- **G3 Riproducibilità:** assenza di **determinismo** preclude l'integrazione in pipeline CI/CD
- **G4 Oracle problem:** nessun meccanismo sistematico per costruire **criteri di valutazione** automatici





Obiettivi

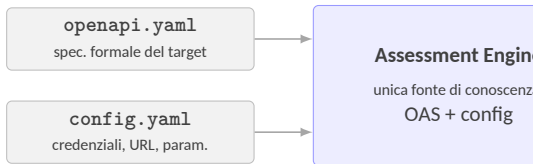
Un tool **contract-driven** per la security assurance
di **qualsiasi API REST documentata**: agnostico, sistematico, riproducibile.

- **O1 Agnosticismo applicativo**: OAS e config.yaml come uniche sorgenti a runtime, nessun riferimento hardcoded al target
- **O2 Portabilità con profondità semantica**: superare il compromesso tramite il paradigma contract-driven
- **O3 Riproducibilità deterministica**: output stabile e integrabile in pipeline CI/CD
- **O4 Specified oracles**: criteri di valutazione automatici derivati dalla conoscenza del dominio



Design contract-driven

- **Paradigma contract-driven:** la specifica **OAS** guida la costruzione di probe **semanticamente valide** e delimita la superficie di assessment
- **Agnosticismo applicativo:** le uniche sorgenti di conoscenza del target a runtime sono **openapi.yaml** e **config.yaml**
 - nessun riferimento hardcoded al target nel codice
 - deployabile su **qualsiasi API REST documentata** senza modifiche





Tassonomia: otto domini di security assurance

- **Do API Discovery:** shadow API, inventario endpoint
- **D1 Authentication:** JWT, trasporto credenziali, revoca
- **D2 Authorization:** RBAC, BOLA, segregazione orizzontale e verticale
- **D3 Data Integrity:** coerenza payload, HMAC
- **D4 Availability:** rate limiting, circuit breaker
- **D5 Observability** [0 test attivi]
- **D6 Config & Hardening:** gateway, header HTTP
- **D7 Business Logic:** SSRF, race condition

18 test attivi su **7 domini**; D5 architettralmente preparato, nessun test in questa versione



Box gradient: profondità di assessment

Il livello di ogni test è una **conseguenza delle precondizioni di accesso disponibili**, non una tassonomia imposta dall'esterno.

BLACK_BOX

Nessun prerequisito

Simulazione attaccante esterno
senza credenziali; controlli
perimetrali del gateway

GREY_BOX

Token per ≥ 2 ruoli

Garanzie che presuppongono
stato autenticato; difetti RBAC,
BOLA, segregazione

WHITE_BOX

Admin API del gateway

Ispezione diretta della
configurazione tramite piano di
controllo

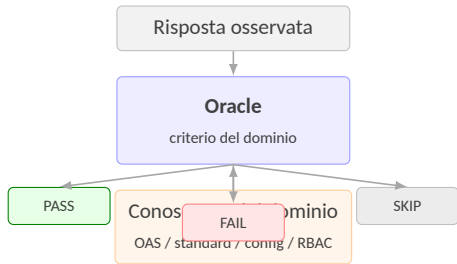


Specified oracles: il criterio di valutazione

Come si stabilisce automaticamente se il comportamento osservato è una violazione?

- Il problema dell'oracolo (Barr et al. 2015): generare input è risolto; **valutare il comportamento osservato** è il collo di bottiglia
- APIGuard adotta **specified oracles**: criteri codificati **a priori** nella logica di ogni test, ricavati dalla conoscenza del dominio del controllo
- Sorgenti per dominio:

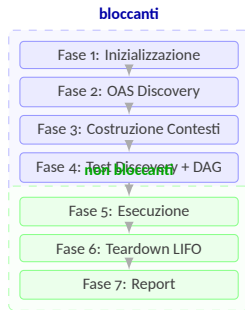
schema OAS → conformità strutturale





Pipeline di esecuzione e connector

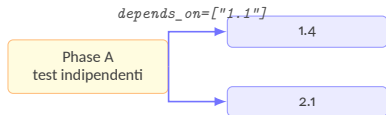
- Pipeline a **7 fasi sequenziali**: prime 4 bloccanti (errore interrompe l'avvio), ultime 3 non bloccanti (errori isolati, pipeline prosegue)
- Connector **gerarchici** a 3 livelli: separano la raccolta dei dati grezzi dalla valutazione dell'esito; lo stesso connector funziona in locale, CI/CD e ambienti containerizzati
- Discovery **dinamico**: nuove verifiche vengono registrate automaticamente senza modifiche al core





DAG scheduler e integrazione CI/CD

- Le dipendenze tra test sono dichiarate **esplicitamente** e risolte **topologicamente**; nessun test può essere eseguito prima dei suoi prerequisiti
- Senza DAG, i test RBAC eseguiti prima di quelli di autenticazione producono errori non deterministici invece di fallimenti puliti



Exit codes semantici per CI/CD

- 0 CLEAN** — tutti PASS/SKIP: pipeline prosegue
- 1 FAIL** — almeno un FAIL: blocco merge/deploy
- 2 ERROR** — malfunzionamento interno: revisione manuale
- 10 INFRA** — errore infrastrutturale: diagnosi deployment



Validazione sperimentale

Testbed

- Forgejo 14.0.3 esposto tramite **Kong 3.9 DB-less** in Docker locale
- Scelto per: OAS nativa, autenticazione reale con ruoli multipli, endpoint con gradi diversi di protezione
- Assenza di riferimenti a Forgejo nel codice: **conferma pratica dell'agnosticismo**

Discrepanza rilevata

Forgejo dichiara autenticazione globale; alcuni endpoint genuinamente pubblici rispondono senza

Risultati

Test attivi	18 su 7 domini
PASS	9
FAIL	7
SKIP	2
ERROR	0
Finding totali	98

Idempotenza

- 2 run indipendenti: risultati **byte-identici**



Conclusioni

- **G1** risolto: probe semanticamente valide dalla specifica OAS **raggiungono la logica applicativa**
- **G2** risolto: profondità di assessment **mantenuta su qualsiasi API REST documentata**, senza riscritture
- **G3** risolto: output byte-identico tra run successive, **integrabile via exit code** in CI/CD
- **G4** risolto: specified oracles per ogni verifica, **nessuna interpretazione manuale**

La security assurance diventa un processo **continuo e **misurabile****

- Tassonomia a 8 domini applicabile indipendentemente dall'implementazione
- Approccio sistematico agli specified oracles nel security testing REST
- Assessment engine deterministico basato su DAG, verificato empiricamente



Sviluppi futuri

Estensioni operative

- Adapter per **gateway cloud managed** (AWS API Gateway, Azure APIM): oggi privi di Admin API accessibile, copertura limitata a BB e GB
- Completamento dominio **D5 Observability**
- **Parallelizzazione** intra-batch per ridurre il wall-clock

Direzioni di ricerca

- Piattaforma modulare per **protocolli eterogenei** (gRPC, GraphQL)
- **Formalizzazione** degli oracoli di sicurezza
- **Coverage augmentation** progressiva basata sui risultati precedenti



Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

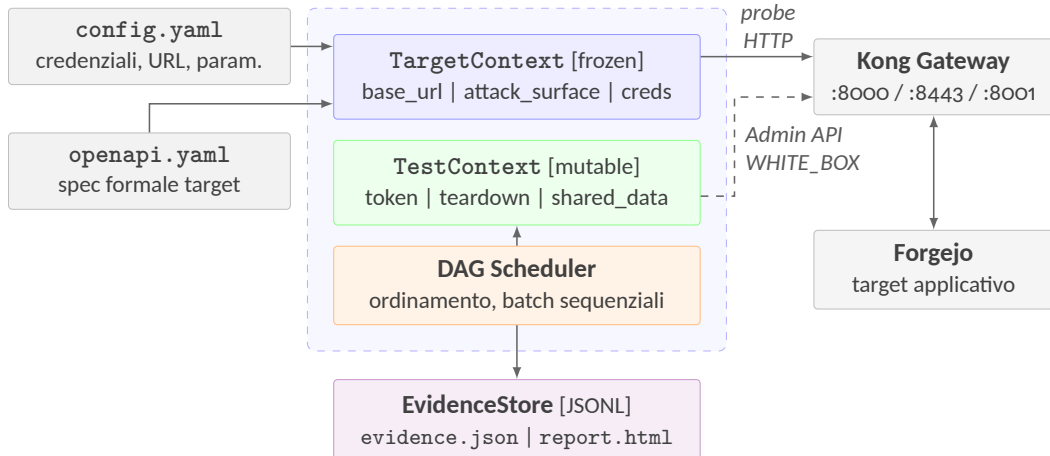
Grazie per l'ascolto! Qualche domanda?

Enea Manzi: enea.manzi@studenti.unimi.it



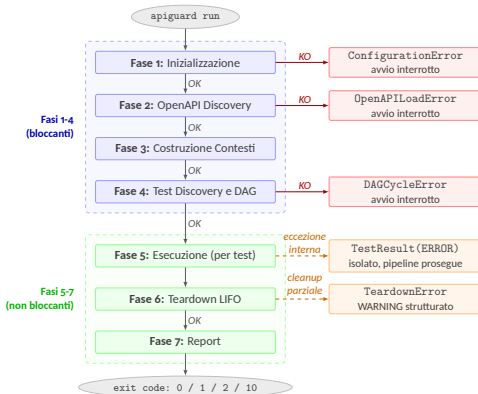
Architettura di APIGuard Assurance

Assessment Engine





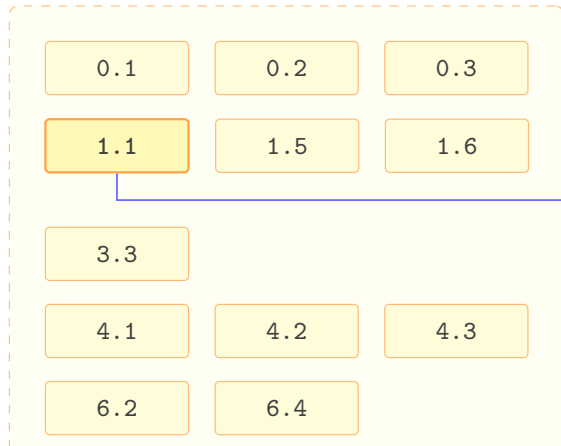
Pipeline a sette fasi



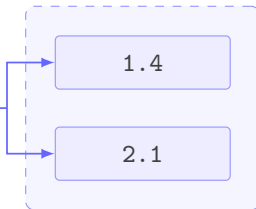


Topologia DAG

Phase A (16 test, nessuna dipendenza)



Phase B (depends_on=["1.1"])





Mappa di copertura (vo.1.0)

